# Accurate Contact-Aware Control of a 2-Link Hopper via Residual Dynamics Learning and MPPI

Vaibhav Mahapatra

***Abstract*—This project presents a hybrid model-based control framework for stabilizing a planar two-link hopping robot using Model Predictive Path Integral (MPPI) control. To accurately capture the system's dynamics, we model the known physics-based coupling dynamics $f(x, u)$ and learn a residual term $g_\theta(x, u)$ to account for complex contact-induced nonlinearities. Residual learning requires representative data in relevant regions of the state-action space. To this end, we train a Proximal Policy Optimization (PPO) [1] agent to generate safe and physically consistent trajectories, ensuring meaningful coverage for dynamics learning. The collected data is then used to fit a neural network to approximate the residual term $g_\theta(x, u) \approx \dot{x} - f(x, u)$. Finally, the learned hybrid dynamics model is embedded within an MPPI controller to generate stabilizing torques that maintain the robot's upright posture. The results demonstrate that augmenting analytical models with learned residuals significantly improves control performance in contact-rich scenarios.**

***Index Terms*—Residual Dynamics Learning, Model Predictive Path Integral (MPPI) Control, Contact-Rich Robotics, Hopping Robot, Hybrid Dynamics Modeling, Reinforcement Learning, Proximal Policy Optimization (PPO), Neural Network Approximation, MuJoCo Simulation, Nonlinear Control Systems**

## I. Introduction

Legged locomotion in robotics presents significant challenges due to the hybrid, underactuated, and contact-rich dynamics inherent in these systems. Accurate modeling of such dynamics is critical for control, yet purely analytical models often fall short in capturing discontinuities and unmodeled interactions during contact events. This motivates a hybrid approach that augments first-principles dynamics with data-driven corrections.

In this work, we focus on a planar two-link hopping robot simulated in MuJoCo [2], with the goal of maintaining an upright posture despite nonlinear ground contact dynamics. To address the modeling gap, we decompose the system dynamics as

$$\dot{x} = f(x, u) + g_\theta(x, u),$$

where $f(x, u)$ represents the known physics-based coupling dynamics, and $g_\theta(x, u)$ is a learned residual term that captures unmodeled effects.

A key challenge in learning residual dynamics lies in collecting meaningful training data. Uniform random actions typically lead to scattered and physically irrelevant state transitions. To constrain data collection within dynamically valid regions, we train a Proximal Policy Optimization (PPO) agent [1] to generate exploratory yet safe trajectories. These trajectories form a reliable dataset of $(x, u, \dot{x})$ tuples for supervised residual learning.

The learned hybrid dynamics model is then used within a Model Predictive Path Integral (MPPI) controller [3], which samples and evaluates noisy control trajectories to select actions that stabilize the robot. MPPI's sampling-based formulation naturally accommodates black-box dynamics, making it ideal for our hybrid model.

Our results demonstrate that augmenting analytical models with residual learning leads to significantly improved performance in tasks involving sharp contact transitions. The approach also generalizes well to variations in initial conditions, highlighting the robustness of the hybrid model-predictive control pipeline.

## II. Robot Dynamical System

This section describes the physical structure of the planar hopper robot, its interactions with the environment, and the key components of the simulation used to train and control the system.

### A. Shape and Inertia Properties

The robot is modeled as a planar two-link system comprising a base and an upper body (torso), connected via a single actuated revolute joint at the hip. The base body is constrained to translate in the vertical and horizontal directions and rotate about the out-of-plane axis, while the torso rotates relative to the base.

The robot's inertial and geometric parameters are summarized below:

- Link lengths: $l_1 = l_2 = 0.3\,\mathrm{m}$
- Masses: $m_1 = m_2 = 1.0\,\mathrm{kg}$
- Moments of inertia: $I_1 = I_2 = 0.01\,\mathrm{kg\,m^2}$

These values are used in computing the mass matrix, Coriolis terms, and gravity forces for the analytical model $f(x, u)$.

### B. Interaction Schematic

The robot interacts with the environment primarily through contact between the lower capsule-shaped body and the ground plane. Contact forces are modeled using MuJoCo's soft-contact formulation, with parameters such as friction and restitution defined in the XML description.

- The robot is affected by gravity ($g = 9.81\,\mathrm{m/s^2}$).
- Ground-plane interactions include friction in both tangential and normal directions.
- Wrap-around logic ensures the robot stays within a finite simulation window and avoids drifting out of view.

A schematic of the robot and its world interaction through external forces is shown in Figure 1. The robot experiences a

normal force as well as friction force on contact. Note, the $\tau$ in the hip refers to the joint actuation that the robot is built with.

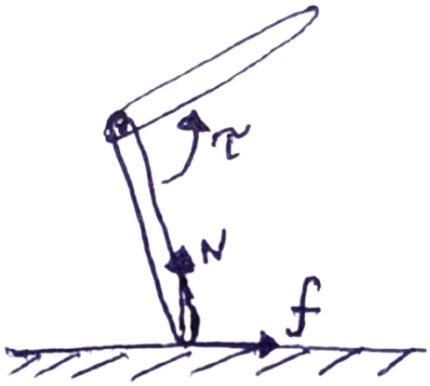



Fig. 1: Robot's interaction with the world

### C. Simulation Environment

The simulation is implemented using the MuJoCo [2] physics engine within a custom OpenAI Gym-compatible environment. The environment encapsulates the robot's dynamics, control interface, reward computation, and logging for data collection.

*a) State Space::* The state vector is an 8-dimensional continuous state:

$$\mathbf{x} = [x, z, \theta, \dot{x}, \dot{z}, \dot{\theta}, \phi, \dot{\phi}]^T$$

where $x, z$ are base positions, $\theta$ is base orientation, and $\phi$ is the hip joint angle. Fig. 2 illustrates these states.

*b) Action Space::* The action is a scalar torque applied at the hip joint:

$$u \in [-10, 10]$$

Actions are bounded and clipped at runtime for simulation safety.

*c) Reward Function::* The reward is designed to encourage upright posture and vertical elevation of the torso. A future section will discuss the exact balance of these factors, but the main ones are described below:

- Upright reward: proportional to $\cos(\theta_{\text{torso}})$, where $\theta_{\text{torso}} = \theta + \phi$
- Alive bonus: fixed constant when torso is above threshold height
- Penalties: hip overuse penalty and termination if torso falls below 0.35 m or exceeds tilt threshold

The environment logs trajectories of $(x, u, x')$ transitions, enabling downstream training of a neural residual model to learn dynamics beyond the analytical formulation.

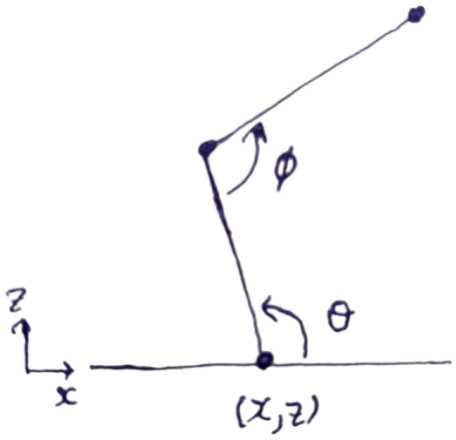



Fig. 2: Robot's state space for learning

## III. Reinforcement Learning for Upright Stabilization

To bootstrap data collection for residual dynamics learning, we first train a reinforcement learning (RL) agent to balance the robot upright. A well-performing policy enables controlled exploration in dynamically valid regions of the state space, ensuring physically meaningful and safe trajectories.

### A. Choice of Algorithm

We considered several candidate algorithms for continuous control in simulation. Table I compares the key properties of popular model-free methods.

TABLE I: Comparison of RL algorithms for continuous control

| Algorithm | On-policy | Exploration | Stability |
|---|---|---|---|
| PPO | Yes | Clipped surrogate loss | High |
| A2C | Yes | Entropy-regularized | Medium |
| SAC | No | Entropy-regularized, off-policy | High |
| TD3 | No | Target smoothing, off-policy | Medium |

We selected Proximal Policy Optimization (PPO) [1] due to its robustness, simplicity, and stability in on-policy learning. Unlike off-policy methods like SAC or TD3, PPO does not require large replay buffers, making it well-suited for environments with moderate stochasticity and consistent contact mechanics. Additionally, PPO's ability to constrain policy updates prevents premature convergence or unstable behavior, which is crucial during early training when falling over is common.

For the subsequent part of the reward shaping, I kept the following parameters of the PPO agent consistent for effective benchmarking:

1) Policy architecture - Multilayer Perceptron

2) Total timesteps - 1,000,000
3) Batch size - 64
4) Training epochs - 10
5) Discount factor ($\gamma$) - 0.99

### B. Reward Shaping Strategies

Designing an effective reward function was key to guiding the agent toward stable hopping. Below, we list the various strategies explored. A key thing to note here is that in Table II, I've listed the most prominent and notable runs. I trained more than 45 agents with different rewards with structures similar to those mentioned here, but with different weights contributing towards the total reward. The top reward in the table worked the best. Key observations from this results include:

- The hip penalty encourages smaller control inputs or torques.
- A height reward is only given when the robot is above the designated 0.4 height.
- The best robot manages to stay upright significantly longer than the other ones. The main factor seems to be the height incentive. Future agents can be trained with more emphasis on this reward factor.

These reward structures were iteratively tested and refined to encourage upright posture, discourage excessive oscillation, and promote long survival times.

### C. Data Collection for Downstream Modeling

Once the PPO agent attained reasonable stability in maintaining upright balance, we leveraged it as a data generator to collect high-quality state transitions. These trajectories serve as the training data for the residual dynamics model that learns to approximate contact-induced effects beyond the analytical model.

*a) Logging Transitions:* The custom Gym-compatible environment was instrumented to record each interaction in the form of $(x_t, u_t, x_{t+1})$ triplets. Specifically, after each call to the `step()` function, the environment logs the current observation vector $x_t \in R^8$, the scalar action $u_t$, and the resulting next observation $x_{t+1}$. This logging is implemented inside the environment's internal logic and is automatically reset when episodes terminate.

*b) Computing $\dot{x}$:* Since MuJoCo internally integrates the equations of motion, direct access to $\dot{x}$ is not available. Instead, we approximate the state derivative using finite differences:

$$\dot{x}_t \approx \frac{x_{t+1} - x_t}{\Delta t}$$

where $\Delta t$ is the effective simulation timestep, computed as:

$$\Delta t = \texttt{sim_steps_per_step} \times \texttt{model.opt.timestep}$$

This gives a reasonably accurate estimate of the state derivative under the assumption of high control frequency and smooth policy behavior.

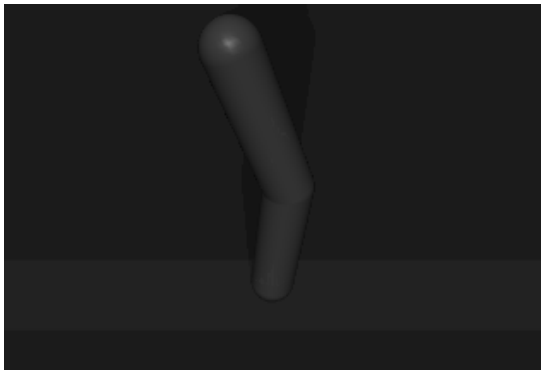


Fig. 3: Snippet of the best PPO Agent at work

*c) Dataset Structure:* Each logged data point is structured as a tuple $(x_t, u_t, \dot{x}_t)$. Once collected, the entire dataset is stored in memory or serialized for disk storage as NumPy arrays for the future training pipeline used. This dataset is later used to train a neural network to approximate the residual term $g_\theta(x, u) \approx \dot{x} - f(x, u)$, where $f(x, u)$ is the output of the analytical dynamics model.

*d) Coverage and Diversity:* Using a PPO policy for exploration ensures that the collected trajectories remain within dynamically feasible and relevant regions of the state space. This avoids the pitfall of uniform random exploration, which may induce non-physical transitions and lead to poor generalization in the residual model.

The best agent is able to stay upright for significant durations. A snippet of the agent can be seen in Fig. 3. A few gifs showing the performance of my best agent can be found in this link: PPO_agent.gif

## IV. Modeling the System Dynamics

To accurately capture the robot's motion, we split the overall dynamics model into two components: an analytical model $f(x, u)$ that captures the coupling mechanics, and a learned residual model $g_\theta(x, u)$ that accounts for contact and unmodeled nonlinearities.

### A. Coupling Mechanics $f(x, u)$

We derive the base dynamics using the Euler-Lagrange method. The generalized coordinates are:

$$q = [x, z, \theta, \phi], \quad \dot{q} = [\dot{x}, \dot{z}, \dot{\theta}, \dot{\phi}]$$

*a) Kinetic Energy (with values plugged in):*

$$T = 1.0(\dot{x}^2 + \dot{z}^2) + 0.005\dot{\theta}^2 + 0.005(\dot{\theta} + \dot{\phi})^2$$

*b) Potential Energy:*

$$V = 19.62z + 0.44\cos(\theta) + 1.47\cos(\theta + \phi)$$

*c) Final Formulation:*

$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{q}}\right) - \frac{\partial \mathcal{L}}{\partial q} = \tau$$

Using the Lagrangian equation, we obtain:

$$M(q)\ddot{q} + C(q, \dot{q}) + G(q) = \tau, \quad \tau = [0, 0, 0, u]$$

| Reward shape | Mean Reward | Max Reward | Mean Ep. Length | Mean $\cos(\theta_{\text{torso}})$ | Mean $\tau$ |
|---|---|---|---|---|---|
| $\cos(\theta_{\text{torso}}) + 10 \cdot \max(0, z-0.4)^2 - 0.1\lvert\phi\rvert$ | 712.24 | 1928.12 | 357.07 | -0.534 | 0.681 |
| $\cos(\theta_{\text{torso}}) + \max(0, z-0.4)^2 - 0.1\lvert\phi\rvert$ | 179.23 | 329.53 | 89.82 | -0.650 | 0.559 |
| $\cos(\theta_{\text{torso}}) + \max(0, z-0.4) - 0.1\lvert\phi\rvert$ | 177.48 | 333.46 | 88.95 | -0.633 | 0.577 |
| $\cos(\theta_{\text{torso}}) + \max(0, z-0.4)^2$ | 163.80 | 311.70 | 82.06 | -0.651 | 0.669 |
| $\cos(\theta_{\text{torso}}) + \max(0, z-0.4)$ | 163.97 | 319.70 | 82.14 | -0.589 | 0.658 |
| $\cos(\theta) + \cos(\phi)$ | 330.88 | 596.35 | 110.46 | -0.793 | 1.218 |
| $\cos(\theta)$ | 178.98 | 339.61 | 89.70 | -0.710 | 0.519 |

TABLE II: Comparison of Reward Shaping Strategies and Performance Metrics

*d) Implementation Details:* This physics was implemented in two ways:

- **Finite Difference:** Approximated Jacobians via perturbation. While straightforward, it was noisy and sensitive to small $\epsilon$.
- **Symbolic Derivation (via SymPy):** Clean, fast, and numerically stable. We analytically derived the matrix expressions for $M$, $C$, and $G$, hardcoded them, and used them in simulation.

Given the sensitivity of contact-rich dynamics, we chose the symbolic approach for the final implementation.

### B. Contact Physics: Learned Residual $g_\theta(x, u)$

The analytical model doesn't capture contact impulses and frictional effects. We model these using a neural network that learns the residual:

$$g_\theta(x, u) \approx \dot{x} - f(x, u)$$

The method is inspired by the work of Nagabandi et Al. [4]

*a) Architecture:* The input is $[x, u]^T \in R^9$, and output is an 8D correction:

$$g_\theta : R^9 \to R^8$$

The hyperparameters of the neural network that's fit to this data are:

- Hidden layers: 3
- Hidden units: (128, 128, 128)
- Activation: Rectified Linear Unit (ReLU)
- Optimizer / LR: Adam (lr = 1e-3)
- Epochs: 50

It takes roughly 3 mins per epoch, giving up a total training time of 2.5 hours.

*b) Training:* Using collected data $(x_i, u_i, \dot{x}_i)$, we defined the target as:

$$y_i = \dot{x}_i - f(x_i, u_i)$$

Loss function:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_i \left\| g_\theta(x_i, u_i) - y_i \right\|^2$$

*c) Evaluating the quality of fit:* The metric performance is as follows:

- Training mean squared error (MSE) = 0.383539
- Validation MSE = 0.207662
- $R^2$ = 0.9899

From Fig. 4, we can see that the model trains till convergence, without any overfitting. Additionally, we can see in Fig. 5 that the residuals are being approximated fairly well. In parity plots, if the trend is along the y=x line, the predicted values are closely lining up with the true values

## V. MPPI Control Using Hybrid Dynamics

With the hybrid dynamics model $\dot{x} = f(x, u) + g_\theta(x, u)$ in place, we implement a Model Predictive Path Integral (MPPI) [3] controller to stabilize the hopper and maintain upright balance. MPPI is a sampling-based model predictive control algorithm that optimizes actions through stochastic trajectory rollouts, making it particularly effective for systems with nonlinear and black-box dynamics.

### A. Cost Function Design

The cost function is critical in guiding the sampling process. We design it to penalize deviations from upright posture, low height, and high control effort. The instantaneous cost at time step $t$ for state $x_t$ and action $u_t$ is defined as:

$$c(x_t, u_t, t) = 10 \cdot \theta_{\text{torso}}^2 + 2 \cdot (z - 0.5)^2 + 0.01 \cdot u_t^2$$

where:

- $\theta_{\text{torso}} = \theta + \phi$: Combined torso angle (base + hip)
- $z$: Vertical position of the base
- $u_t$: Control input (hip torque)

This structure encourages upright balance, maintains a minimum height threshold, and discourages excessive torque usage.

### B. Trajectory Rollouts

At each control step, MPPI samples $K$ candidate action sequences $\{\mathbf{u}_{0:T-1}^{(k)}\}_{k=1}^K$ by injecting noise into the current nominal control sequence $\mathbf{u}_{0:T-1}$. Each action sequence is clipped within actuator limits and rolled out through the hybrid dynamics model.

For t = 0, 1, .. T-1, The rollout process for sample $k$ proceeds as:

$$x_0^{(k)} = x \quad \text{(initial state)}$$

$$x_{t+1}^{(k)} = x_t^{(k)} + \Delta t \cdot \left[ f(x_t^{(k)}, u_t^{(k)}) + g_\theta(x_t^{(k)}, u_t^{(k)}) \right]$$

The total trajectory cost is accumulated as a sum of instantaneous costs:

$$S_k = \sum_{t=0}^{T-1} c(x_t^{(k)}, u_t^{(k)}, t)$$




Fig. 4: Training and Validation Losses while training $g_\theta(x, u)$

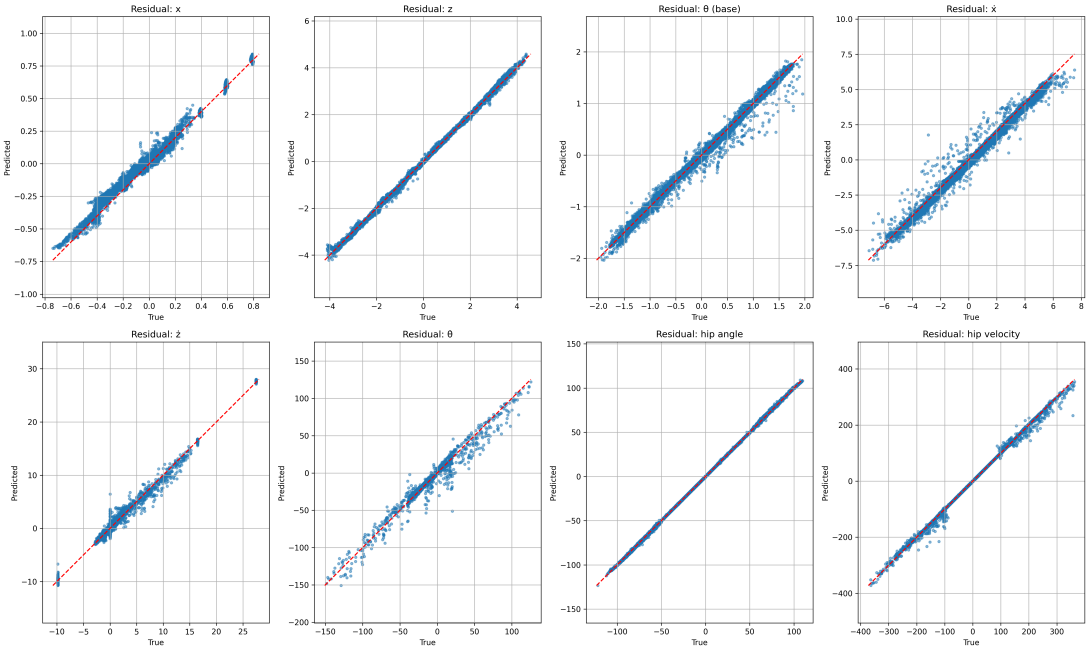



Fig. 5: Parity plot of each state's residuals

### C. Control Selection

After computing costs $S_k$ for all sampled trajectories, MPPI performs a softmin weighting to compute the optimal update to the nominal control sequence:

$$\beta = \min_k S_k$$

$$w_k = \frac{\exp\left(-\frac{1}{\lambda}(S_k - \beta)\right)}{\sum_{j=1}^{K} \exp\left(-\frac{1}{\lambda}(S_j - \beta)\right)}$$

$$\mathbf{u}_{0:T-1} \leftarrow \sum_{k=1}^{K} w_k \cdot \mathbf{u}^{(k)}_{0:T-1}$$

The first control $u_0$ from the updated sequence is applied to the environment:

$$u_{\text{applied}} = \mathbf{u}_0$$

and the sequence is shifted for the next timestep (receding horizon):

$$\mathbf{u}_{0:T-1} \leftarrow [\mathbf{u}_1, \ldots, \mathbf{u}_{T-1}, \mathbf{0}]$$

### D. Implementation and Numerical Stability Issues

The MPPI controller was implemented using the hybrid dynamics model as the forward rollout function. However, despite correct formulation and rollout logic, the simulation encountered numerical instability - notably, the state vector began returning `NaN` values shortly after control execution.

This instability prevented full deployment and evaluation of the MPPI controller in simulation. A few potential causes for these numerical issues are:

- **Excessive Control Magnitudes:** Samples generated by MPPI occasionally exceed stable torque bounds, even

after clipping. Large torques can drive the system into unmodeled configurations or cause MuJoCo to enter invalid simulation states.

- **Stiff Dynamics and Integration Error:** The residual model introduces sharp corrections that may destabilize integration when combined with high-frequency sampling. This is especially sensitive in contact-rich regimes with hybrid dynamics.
- **Invalid Inputs to Residual Network:** Out-of-distribution inputs during trajectory sampling (e.g., extreme poses) could cause the neural network to produce unreliable or diverging predictions.

*a) Possible Fixes::*

- Tighten control bounds and resample only within safe operational regions.
- Add gradient clipping or soft activation bounds in the residual network.
- Normalize inputs and apply input validation during MPPI rollouts.
- Add a fallback to purely analytical dynamics if residual predictions exceed a threshold.

Future work will involve identifying the precise source of instability through unit tests on the dynamics rollout loop, validating residual network outputs for extreme samples, and stabilizing rollouts via control smoothing or adaptive step-size integration.

## VI. Conclusions and Future Work

The overarching objective of this project was to model the dynamics of a robotic system subject to contact-induced nonlinearities, and to leverage this model within a model predictive control (MPC) framework — specifically, Model Predictive Path Integral (MPPI) control — to achieve upright balance in a 2D hopper robot.

While the initial focus was on learning residual dynamics over an analytical baseline, the project naturally evolved to first address the challenge of data quality. Rather than relying on random exploration, which often leads to unrealistic or unstable transitions, we trained a model-free reinforcement learning (RL) agent using Proximal Policy Optimization (PPO) to generate safe, physically meaningful trajectories. This detour proved crucial in enabling the residual learning pipeline and exposed me to the nuances of reward shaping, episode management, and training dynamics across multiple RL strategies.

Once the PPO agent demonstrated competent behavior in balancing the robot, we successfully collected structured state-action-next-state triplets and fit a residual neural network atop a Lagrangian-based physics model. The hybrid dynamics formulation showed good regression performance, indicating that residual learning can meaningfully capture contact effects absent in the analytical model.

The MPPI controller was fully implemented and theoretically integrated with the hybrid dynamics model. However, numerical instabilities during rollout (e.g., NaN values in state trajectories) prevented successful real-time control. These issues are likely due to sharp discontinuities in residual predictions, integration instability, or out-of-distribution samples during trajectory sampling. Addressing these remains a key area for improvement.

*Future Directions*

Several promising avenues exist for extending this work:

- **Stabilizing MPPI Execution:** Introduce regularization in the cost function, better input normalization for the residual model, or fallback dynamics to improve numerical robustness.
- **Exploration of Alternative Controllers:** Test other MPC frameworks such as iLQR or Cross-Entropy Method (CEM), or evaluate hybrid planning-RL pipelines.
- **Adaptive Residual Learning:** Investigate online learning techniques that adapt the residual model during rollouts, allowing corrections in response to newly visited states.
- **High-Fidelity Contact Modeling:** Use more expressive architectures (e.g., mixture density networks or GNNs) to learn finer-grained contact behavior.
- **Task Generalization:** Extend from balancing to hopping or trajectory tracking, introducing more challenging dynamics while maintaining the core modeling framework.
- **Hardware Transfer:** Investigate the feasibility of sim-to-real transfer with a physical 2D hopper prototype, using learned dynamics as a prior or regularizer.

This project provided a comprehensive view of modern control pipelines combining physics modeling, data-driven learning, and RL-based bootstrapping. It highlighted both the power and the intricacies of integrating model-based and model-free methods in a contact-rich robotics setting.

## References


[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," in *arXiv preprint arXiv:1707.06347*, 2017.

[2] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5026–5033, 2012.

[3] G. Williams, P. Drews, B. Goldfain, J. Rehg, and E. Theodorou, "Aggressive driving with model predictive path integral control," in *2016 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2016, pp. 1433–1440.

[4] A. Nagabandi, G. Kahn, R. S. Fearing, and S. Levine, "Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 7559–7566.